

```
In [1]: import pennylane as qml
from pennylane import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import pandas as pd
```

```
In [3]: df=pd.read_csv('../Datasets For Classification/Bike Sharing/day.csv')
df
```

```
Out[3]:
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit
0	1	2011-01-01	1	0	1	0	6	0	2 0.0
1	2	2011-01-02	1	0	1	0	0	0	2 0.0
2	3	2011-01-03	1	0	1	0	1	1	1 0.0
3	4	2011-01-04	1	0	1	0	2	1	1 0.0
4	5	2011-01-05	1	0	1	0	3	1	1 0.0
...	...	...	...	...	...	...	...	...	...
726	727	2012-12-27	1	1	12	0	4	1	2 0.0
727	728	2012-12-28	1	1	12	0	5	1	2 0.0
728	729	2012-12-29	1	1	12	0	6	0	2 0.0
729	730	2012-12-30	1	1	12	0	0	0	1 0.0
730	731	2012-12-31	1	1	12	0	1	1	2 0.0

731 rows × 16 columns



```
In [10]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, accuracy_score, precision_score
from sklearn.preprocessing import KBinsDiscretizer

import pennylane as qml
from pennylane.optimize import NesterovMomentumOptimizer
import matplotlib.pyplot as plt

# 1. Load Dataset
```

```

bike_df = pd.read_csv("../Datasets For Classification/Bike Sharing/day.csv")

# 2. Preprocessing
def preprocess(df):
    df = df.copy()
    features = ['temp', 'atemp', 'hum', 'windspeed', 'season', 'holiday', 'worki
    X = df[features]

    # Convert target to classification (e.g., Low=0, medium=1, high=2)
    y_raw = df['cnt'].values.reshape(-1, 1)
    discretizer = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='quantil
    y = discretizer.fit_transform(y_raw).astype(int).ravel()

    # Scaling
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Train-test split
    return train_test_split(X_scaled, y, test_size=0.2, random_state=42)

X_train, X_test, y_train, y_test = preprocess(bike_df)

# 3. Define Quantum Circuit
n_qubits = X_train.shape[1]
dev = qml.device("default.qubit", wires=n_qubits)

def angle_encoding(x, wires):
    for i in range(len(x)):
        qml.RY(x[i], wires=wires[i])

def variational_circuit(weights, wires):
    for i in range(n_qubits):
        qml.RY(weights[i], wires=wires[i])
    for i in range(n_qubits - 1):
        qml.CNOT(wires=[wires[i], wires[i + 1]])

@qml.qnode(dev)
def circuit(weights, x):
    angle_encoding(x, wires=range(n_qubits))
    variational_circuit(weights, wires=range(n_qubits))
    return [qml.expval(qml.PauliZ(i)) for i in range(n_qubits)]

# 4. VQC Classifier Logic
def predict(weights, X):
    predictions = []
    for x in X:
        x_padded = np.pad(x, (0, n_qubits - len(x))) # zero-pad to match qubits
        result = circuit(weights, x_padded)
        summed = np.sum(result)
        # Map to class
        if summed < -0.5:
            predictions.append(0)
        elif summed < 0.5:
            predictions.append(1)
        else:
            predictions.append(2)
    return np.array(predictions)

# 5. Training Loop
def cost(weights, X, y):

```

```

    y_pred = predict(weights, X)
    return np.mean((y_pred - y) ** 2)

np.random.seed(42)
weights = np.random.randn(n_qubits)
opt = NesterovMomentumOptimizer(stepsize=0.1)

steps = 20
for i in range(steps):
    weights, _ = opt.step_and_cost(lambda w: cost(w, X_train, y_train), weights)
    if i % 5 == 0:
        print(f"Step {i}: Cost = {cost(weights, X_train, y_train):.4f}")

# 6. Evaluate
y_pred = predict(weights, X_test)
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

metrics = {
    'accuracy': accuracy_score(y_test, y_pred),
    'precision': precision_score(y_test, y_pred, average='weighted'),
    'recall': recall_score(y_test, y_pred, average='weighted'),
    'f1': f1_score(y_test, y_pred, average='weighted')
}
print("Metrics Summary:", metrics)

```

c:\Users\Mr. Nitin\Desktop\quantum_ML\lib\site-packages\pennylane_grad.py:216: UserWarning: Attempted to differentiate a function with no trainable parameters. If this is unintended, please add trainable parameters via the 'requires_grad' attribute or 'argnum' keyword.

warnings.warn(

Step 0: Cost = 1.4949

Step 5: Cost = 1.4949

Step 10: Cost = 1.4949

Step 15: Cost = 1.4949

Classification Report:

	precision	recall	f1-score	support
0	0.10	0.02	0.03	60
1	0.18	0.17	0.18	40
2	0.25	0.53	0.34	47
accuracy			0.22	147
macro avg	0.18	0.24	0.18	147
weighted avg	0.17	0.22	0.17	147

Metrics Summary: {'accuracy': 0.22448979591836735, 'precision': 0.17168100626747243, 'recall': 0.22448979591836735, 'f1': 0.16999770955599863}